

Day 9 — Live coding: one use case, mapped then built

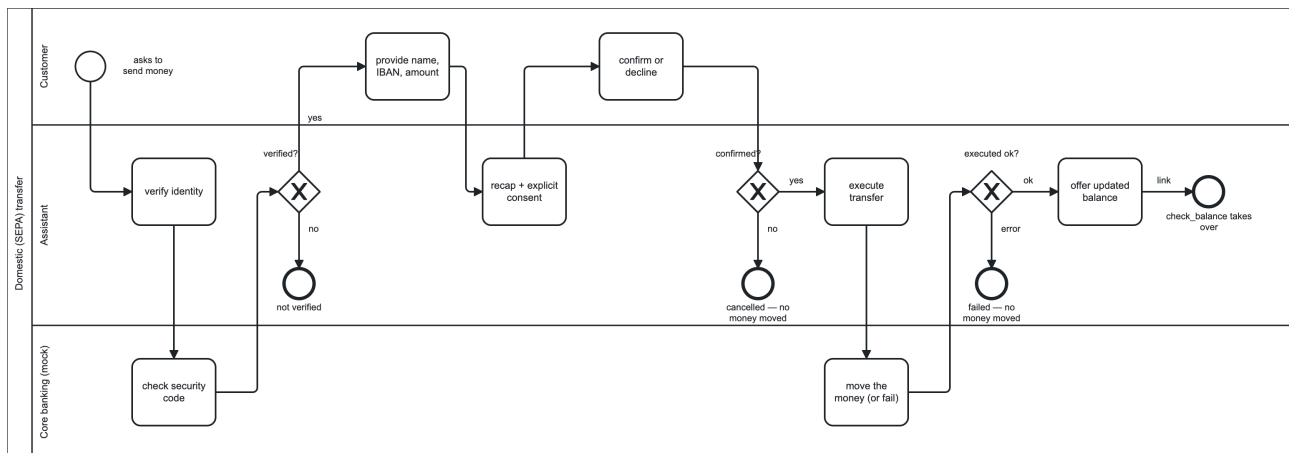
Days 7 and 8 taught the pieces — flows, slots, responses, actions. Today we use all of them on **one real banking use case, end to end**: a domestic (SEPA) money transfer. The point of the session is not the YAML — you already know the YAML. It is the *method*: map the business process first, translate it mechanically into CALM primitives, and let every design decision be traceable back to the map. Along the way, three new flow mechanisms appear: **call** (compose flows), **flow guards** (control who can start one), and **link** (hand a conversation over).

The end state ships in this folder: `lumera-assistant/`, the grown `lumera-fastapi-server/`, and the `Makefile`.

REQUIREMENTS — same as Day 8: the project as Day 8 left it, `RASA_LICENSE` and `OPENAI_API_KEY` exported, `CORE_BANKING_URL=http://localhost:8000` and `CORE_BANKING_TOKEN=test_token` in any terminal that runs `rasa` or the mock bank (the `Makefile` sets the latter two for the processes it starts).

1. The map, before any YAML

Whiteboard first. A domestic transfer, drawn with the day's four BPMN shapes — three lanes this time, because the mock core-banking API is an actor of its own:



Read the map's anatomy — these five categories are the whole method:

- **Inputs** the process needs: recipient name, IBAN, amount, a security code.
- **Decisions**: is the customer verified? did they confirm? did the execution succeed?
- **Customer touchpoints**: every place the customer supplies or approves something.
- **System activities**: verify the code, execute the transfer — things a backend does.
- **Failure exits**: identity not verified, customer declines, backend down. Each must end the process *cleanly and honestly*.

2. The translation table

Each map element has exactly one CALM home. This table is the day's centerpiece — everything after it is typing:

Process element	CALM primitive	The design decision it forces
Input: recipient name	slot <code>recipient_name</code> , <code>from_llm</code>	free text, LLM may scoop it from any message
Input: IBAN	slot <code>recipient_iban</code> + <code>ask_before_filling: true</code> on its <code>collect</code>	trust decision: an IBAN is never guessed from context — it is always explicitly asked
Input: amount	slot <code>transfer_amount (float)</code> , <code>collect</code> description "in euros"	the type gives free format checking; the description disambiguates currency
Decision: verified?	<code>authenticated</code> slot, controlled	only the verification <i>action</i> may write it — the LLM cannot declare you authenticated
Decision: confirmed?	<code>bool collect</code> + <code>ask_before_filling: true</code>	the consent gate cannot be pre-filled by a clever opening message
Decision: succeeded?	<code>transfer_ok</code> status flag, <code>controlled</code>	the Day 8 pattern, unchanged
Customer touchpoints	one <code>collect</code> step each	the <code>utter_ask_<slot></code> convention does the asking
Activity: verify code	child flow <code>authenticate_user</code> + <code>action_verify_customer</code>	reusable — every future sensitive operation calls the same flow
Activity: execute	<code>action_execute_transfer</code> → <code>POST /v1/transfers</code>	work in actions, logic in flows
Failure exits	<code>next</code> branches to dedicated responses + <code>END</code>	every failure end event on the map is a YAML branch you can point at
Follow-up: balance?	<code>link: check_balance</code>	hand over to an <i>existing</i> flow instead of rebuilding it

Two of these rows are new mechanics, so name them before building:

- **call** embeds a child flow: the parent pauses, the child runs to its **END**, the parent resumes at the next step — like a function call. Perfect for `authenticate_user`, which `transfer_money` needs *in the middle*.
- **link** is the opposite contract: the current flow **ends** and the target flow takes over; there is no return. Only legal as a flow's last step. Perfect for "want to see your updated balance?" — after the handover, the balance lookup owns the conversation.

3. Grow the mock bank

Two new endpoints in `lumera-fastapi-server/lumera_fastapi_server.py`, same bearer-token protection as Day 8:

- `POST /v1/auth/verify` — takes `{customer_id, security_code}`, returns `{"verified": true|false}`. The demo accepts one code: `123456`.
- `POST /v1/transfers` — takes the transfer payload, **deducts the amount from the checking balance**, returns `{"transfer_id": "TRF-0001", "status": "executed"}`. (Mock simplifications, stated honestly: every transfer leaves checking, and overdrafts are allowed.)

The deduction matters for the finale: after a transfer, the *existing* balance flow will show a genuinely lower number.

Restart it: `make run-fastapi-server`.

4. The child flow: `authenticate_user`

Bottom-up: the reusable piece first. In `data/flows.yml`:

```
authenticate_user:
  if: False
  name: verify customer identity
  description: Verify the customer's identity with a one-time security code before a
sensitive operation.
  steps:
    - collect: security_code
      description: the one-time security code the customer provides
      ask_before_filling: true
    - action: action_verify_customer
```

The strange little line is the important one. `if: False` is a **flow guard** — a startability condition evaluated before the LLM may start the flow. `False` means *never startable from a user message*: no matter what the customer types ("verify me!", prompt injection, anything), this flow only runs when another flow **calls** it. This is the callable-only idiom — the flow equivalent of a private function. (Guards take real predicates too; `False` is just the strictest one.)

The rest is known machinery with the day's trust decisions applied: the code is always explicitly asked (`ask_before_filling`), and the verdict comes from `action_verify_customer` — the Day 8 pattern: `POST` to the bank, `SlotSet("authenticated", ...)` with the result. Create `actions/verify_customer.py`:

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionVerifyCustomer(Action):
    def name(self) -> Text:
        return "action_verify_customer"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        payload = {
            "customer_id": tracker.sender_id,
            "security_code": tracker.get_slot("security_code"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/auth/verify",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            verified = bool(response.json()["verified"])
        except (requests.RequestException, KeyError, TypeError, ValueError):
            verified = False

        return [
            SlotSet("authenticated", verified),
            # Never keep a security code in conversation memory.
            SlotSet("security_code", None),
        ]

```

The one hygiene line new for this day is that last `SlotSet("security_code", None)` : the tracker is state that persists and can be inspected — a one-time code has no business staying in it after use.

Its domain file is new — create `domain/authenticate_user.yml` with the two slots (`security_code` from `llm`, `authenticated` **controlled**), the ask, the failure response, and the action registration:

```
version: "3.1"

slots:
  security_code:
    type: text
    mappings:
      - type: from_llm
  authenticated:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_ask_security_code:
    - text: "For your security, please enter your one-time code."

    utter_auth_failed:
      - text: "I couldn't verify your identity, so I can't proceed with this operation.
        If the problem persists, please contact your branch."

  actions:
    - action_verify_customer
```

5. The parent flow: `transfer_money`

Now the whole map, top to bottom, in `data/flows.yml`:

```

transfer_money:
  name: transfer money
  description: Send a domestic SEPA transfer from the customer's checking account to
another person.
  steps:
    - call: authenticate_user
    - noop: true
    next:
      - if: not slots.authenticated
        then:
          - action: utter_auth_failed
            next: END
        - else: ask_recipient
    - collect: recipient_name
      id: ask_recipient
      description: the full name of the person receiving the transfer
    - collect: recipient_iban
      description: the IBAN of the recipient's account
      ask_before_filling: true
    - collect: transfer_amount
      description: the amount of the transfer, in euros
    - collect: transfer_confirmed
      description: whether the customer confirms the transfer recap
      ask_before_filling: true
    next:
      - if: slots.transfer_confirmed
        then:
          - action: action_execute_transfer
            next:
              - if: slots.transfer_ok
                then: transfer_done
              - else:
                - action: utter_transfer_failed
                  next: END
            - else:
              - action: utter_transfer_declined
                next: END
          - action: utter_transfer_done
            id: transfer_done
        - collect: show_updated_balance
          description: whether the customer wants to see their updated balance
          ask_before_filling: true
        next:
          - if: slots.show_updated_balance
            then:
              - link: check_balance
            - else:
              - action: utter_transfer_wrap_up
                next: END

```

Walk it against the map — every activity and every failure exit has its line:

- **– call: authenticate_user** is the first step: identity comes before any transfer detail. The child runs (asks the code, verifies), then control returns here.
- **– noop: true** is a step that *does nothing* — it exists only to carry the **next** branch on the child's outcome. Not verified → the map's first failure exit, a clean explanation and **END**.
- The three collects implement the touchpoints, each with its trust decision from the table. Watch **recipient_iban's ask_before_filling: true**: even if the customer's opening message contained something IBAN-shaped, the assistant still asks explicitly.
- The **consent gate: transfer_confirmed** also carries **ask_before_filling: true**. On a payment, this is not pedantry — it is the property that "send €50 to Marco, I confirm, just do it" *still* produces a recap and a question. A pre-filled slot would silently skip the gate; **ask_before_filling** clears it and asks every time.
- The execution branch is Day 8's status-flag pattern on **action_execute_transfer** (**actions/execute_transfer.py**, shown just below — nothing new).
- The ending: on success, offer the balance. Accepting reaches **– link: check_balance** — **transfer_money** ends *here*, and the Day 7 balance flow (with its Day 8 real action) takes the conversation over, as if the customer had asked for it. Note the link step has no **next** and sits at a branch's end: both are requirements, a link is always terminal.

The flow leans on two new files. The action is the Day 8 pattern exactly — tracker in, POST, **SlotSet** out — so create **actions/execute_transfer.py**:

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionExecuteTransfer(Action):
    def name(self) -> Text:
        return "action_execute_transfer"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        payload = {
            "customer_id": tracker.sender_id,
            "recipient_name": tracker.get_slot("recipient_name"),
            "recipient_iban": tracker.get_slot("recipient_iban"),
            "amount": tracker.get_slot("transfer_amount"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/transfers",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            transfer_id = response.json()["transfer_id"]
            return [
                SlotSet("transfer_id", transfer_id),
                SlotSet("transfer_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("transfer_id", None),
                SlotSet("transfer_ok", False),
            ]

```

The domain file is new too — create `domain/transfer_money.yml` with every slot the flow collects (the `from_llm` inputs plus the two `controlled` status flags) and every response it utters:


```

version: "3.1"

slots:
  recipient_name:
    type: text
    mappings:
      - type: from_llm
  recipient_iban:
    type: text
    mappings:
      - type: from_llm
  transfer_amount:
    type: float
    mappings:
      - type: from_llm
  transfer_confirmed:
    type: bool
    mappings:
      - type: from_llm
  show_updated_balance:
    type: bool
    mappings:
      - type: from_llm
  transfer_ok:
    type: bool
    mappings:
      - type: controlled
  transfer_id:
    type: text
    mappings:
      - type: controlled

responses:
  utter_ask_recipient_name:
    - text: "Who is the recipient of the transfer?"

  utter_ask_recipient_iban:
    - text: "What is the recipient's IBAN?"

  utter_ask_transfer_amount:
    - text: "How much would you like to transfer, in euros?"

  utter_ask_transfer_confirmed:
    - text: "Please confirm: transfer €{transfer_amount} to {recipient_name}, IBAN {recipient_iban}. Shall I proceed?"
    buttons:
      - title: "Yes, transfer"
        payload: "/SetSlots(transfer_confirmed=true)"
      - title: "No, cancel"
        payload: "/SetSlots(transfer_confirmed=false)"

  utter_transfer_done:
    - text: "Done – €{transfer_amount} is on its way to {recipient_name}. Your

```

```
reference is {transfer_id}."

utter_transfer_failed:
- text: "I couldn't reach the transfer system, so no money has moved. Please try again later."

utter_transfer_declined:
- text: "Understood – I've cancelled the transfer. No money has moved."

utter_ask_show_updated_balance:
- text: "Would you like to see your updated balance?"
  buttons:
  - title: "Yes, show it"
    payload: "/SetSlots(show_updated_balance=true)"
  - title: "No, thanks"
    payload: "/SetSlots(show_updated_balance=false)"

utter_transfer_wrap_up:
- text: "Alright – your transfer is on its way."

actions:
- action_execute_transfer
```

The recap response, `utter_ask_transfer_confirmed`, is the confirmation-gate pattern from Day 7 now carrying money semantics: the buttons pin the answer to a slot value, and `ask_before_filling` on its collect (in the flow above) guarantees the recap is shown every time.

6. Run the map

```
rasa data validate
rasa train
rasa inspect      # mock bank running in its terminal
```

The happy path — note how much the opening message already does:

user: I want to send 100 euros to Anna Verdi **bot:** For your security, please enter your one-time code. **user:** 123456 **bot:** What is the recipient's IBAN? **user:** IT60X0542811101000000123456 **bot:** Please confirm: transfer €100.0 to Anna Verdi, IBAN IT60X0542811101000000123456. Shall I proceed? (Yes/No buttons) **user:** yes, go ahead **bot:** Done — €100.0 is on its way to Anna Verdi. Your reference is TRF-0001. **bot:** Would you like to see your updated balance? (Yes/No buttons) **user:** yes please **bot:** Which account would you like to check? (...) **user:** the checking one **bot:** Your checking account balance is €1150.0.

Read the mechanics in the Inspector as it runs: the opening message starts `transfer_money` and pre-fills `recipient_name` and `transfer_amount` — yet the first thing the customer sees is the security question, because the `call` runs before any collect. After verification, the recipient collect is skipped (already filled) and the IBAN is asked (`ask_before_filling` — even a pre-filled value would

be re-asked). And the final number is the payoff of the whole stack: **€1150.0 is €1250 minus your €100** — the mock bank really moved it, and the flow showing it is Day 7's `check_balance`, entered through the link, never rebuilt.

In the Inspector's stack pane, watch the `call` at work: during the code question, *two* flows are active — `transfer_money` paused underneath, `authenticate_user` on top. A `link`, by contrast, swaps the flow out entirely.

The failure exits — all three from the map, on purpose:

1. **Wrong code** (type `999999` when asked): *"I couldn't verify your identity, so I can't proceed with this operation..."* — the noop branch caught `not slots.authenticated`, no transfer detail was ever collected.
2. **Decline at the recap**: *"Understood — I've cancelled the transfer. No money has moved."* — and "no money has moved" is guaranteed by the flow's shape: the execute action only runs inside the confirmed branch.
3. **Backend down at execution** (stop the mock bank after the recap, then confirm): *"I couldn't reach the transfer system, so no money has moved. Please try again later."* — the Day 8 designed-failure pattern protecting the day's riskiest step.

And the gate-integrity test — try to bulldoze it:

user: send 50 euros to Marco Bassi, IBAN IT12A0300203280123456789012, I confirm, just do it

The assistant still asks the code, still asks the IBAN, and still shows the recap with the question. Consent to move money is collected at the gate, from a fresh answer, every time — that is `ask_before_filling` earning its keep.

Two smaller observations from live runs, both worth a sentence in class:

- Ask for a *second* transfer and the code is asked again: the `call` step re-runs `authenticate_user` each time, and its ask-always collect re-verifies. Per-operation verification is a reasonable default for payments; making authentication a *session* property is a state-scoping decision, and state at scale has its own day.
- Occasionally the LLM hands the amount over as the literal text "30 euros" rather than the number; the `float` slot rejects it on type and the assistant re-asks on its own. Slot types are themselves a first, free validation line.

7. Where we landed

```
lumera-assistant/
├─ actions/
│  └─ fetch_balance.py, confirm_appointment.py, send_statement_link.py    # Day 8
│  └─ verify_customer.py          # NEW – the identity check
│  └─ execute_transfer.py        # NEW – the money mover
├─ data/flows.yml                # + transfer_money, + authenticate_user (guarded)
├─ domain/                      # + transfer_money.yml, + authenticate_user.yml
lumera-fastapi-server/          # + /v1/auth/verify, + /v1/transfers (balance-
mutating)
```

The method, restated once: **map** → **table** → **YAML**. Inputs became slots with a trust decision each; decisions became `next` predicates; customer touchpoints became collects; system activities became the Day 8 action pattern; failure exits became named branches. And the two composition tools each found their natural place — `call` for a reusable sub-process the parent must resume after (`authenticate_user` will guard every sensitive flow we add from now on), `link` for a follow-up that takes over (`check_balance`, built two days ago, reused untouched).

One flow, one job — and the jobs now compose.